

Betriebssysteme 1 | Bsys1

Zusammenfassung

INHALTSVERZEICHNIS

1. Prozessor	2
1.1. Instruktionen	2
1.1.1. Codierung	2
1.1.2. Sequenzen	2
1.1.3. Takt / Zyklus	3
2. Maschinencode	3
2.1. Intel Terminologie	3
2.2. Formate	3
3. Assembler	4
3.1. Spezifikation von Daten	4
3.1.1. Spezifikation von Zahlen	4
3.2. Bits	5
3.3. Offset	5
3.4. Labels	5
3.5. Flat-Form Binaries	6
3.6. Objekt-Dateien	6
3.6.1. Symboltabelle	6

1. PROZESSOR

Der Prozessor und Speicher sind getrennt und sind durch einen Speicherbus verbunden. Dieser umfasst:

Adressbus: enthält Adresse der Speicherzelle, auf die zugegriffen werden soll

Datenbus: enthält Daten, die aus Speicherzelle gelesen wurde bzw. in diese geschrieben werden soll

Steuersignale: die z.B. angeben, ob gelesen oder geschrieben werden soll

Schreibvorgang	Lesevorgang
1) Prozessor legt Adresse auf Adressbus	1) Prozessor legt Adresse auf Adressbus
2) Prozessor legt Daten auf Datenbus	2) Prozessor aktiviert Speicherbus zum Lesen
3) Prozessor aktiviert Speicherbus zum Schreiben	3) Speicher legt Daten auf Datenbus

- Jeder Prozessor enthält eine kleine Menge an Speicher: die **Register**
- Jeder Prozessor besteht aus vielen kleinen Bausteinen, die jeweils eine **Operation** durchführen können
 - Operationen können Register und/oder den Speicherbus lesen und/oder schreiben
 - Operationen können 0 bis n Operanden (Register, Adressen, Daten) haben
 - Die Anzahl und Art von Operationen ist immer prozessor-spezifisch, aber auf den meisten Prozessoren ähnlich

1.1. INSTRUKTIONEN

Eine **Instruktion** ist die Kombination einer Operation mit Operanden, z.B. «Kopiere Wert aus Register x in Register y». `mov y, x`

1.1.1. Codierung

Operationen und Register werden durchnummeriert. Gemeinsam mit Daten- und/oder Adressoperanden ergibt sich eine Bytesequenz.

Beispiel 1: «Addiere 12345678_h zu Register 1» auf Intel 64 Architektur

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5
81 _h	C1 _h	78 _h	56 _h	34 _h	12 _h
Opcode		12345678 _h			

Die Gesamtheit aller möglichen (binären) Instruktionscodes eines Prozessors (Maschine) ist sein Maschinencode und wird vom Hersteller festgelegt.

1.1.2. Sequenzen

Die Sequenzen der Instruktionen können an mehreren Orten liegen:

- Hart-codiert im Prozessor: Sequenz kann nach Produktion des Prozessors nie geändert werden
- Im Hauptspeicher: Sequenz kann beliebig geändert werden, muss über den Speicherbus abgefragt werden

Der Prozessor hat ein spezielles Register, das die Adresse des nächsten Befehls im Hauptspeicher enthält (Befehlszeiger / **instruction pointer** IP / **program counter** PC).

1.1.3. Takt / Zyklus

Der gesamte Computer ist getaktet: Alle Bausteine (auch im Prozessor, im Speicher etc) erhalten ein Takt-Signal (**Clock**)

Über mehrere Takte hinweg führt der Prozessor folgende Schritte aus (**Zyklus**):

- 1) Prozessor fordert Instruktion ab der Adresse an, die im Befehlszeiger steht
- 2) Prozessor decodiert Operation und Operanden aus Instruktion
- 3) Prozessor wählt mit Operation korrespondierenden Baustein aus
- 4) Aktiver Baustein liest ggf. aus den Registern
- 5) Aktiver Baustein führt ggf. Berechnung aus
- 6) Aktiver Baustein schreibt ggf. in die Register
- 7) Prozessor erhöht Befehlszeiger entsprechend der Länge der Instruktion

Diese Schritte können stark parallelisiert werden (**Pipelining**)

2. MASCHINENCODE

Intel 64 Prozessoren basieren auf einer Little-Endian-Architektur. Es gibt 2 Modi: 32 Bit (Legacy) und 64 Bit, befassen uns hier aber nur mit 64 Bit.

2.1. INTEL TERMINOLOGIE

Begriff	Definition
Byte	8 Bit
Word	2 Byte / 16 Bit
Doubleword	4 Byte / 32 Bit, auch DWord
Quadword	8 Byte / 64 Bit, auch QWord
Double Quadword	16 Byte / 128 Bit, auch DQWord

Intel-Prozessoren verwenden als kleinste Einheiten einzelne Bytes (8 Bit). Ein einzelnes Bit muss immer mit dem gesamten Byte verarbeitet werden. Dargestellt werden die Bytes als 2 hexadezimale Stellen s_0 und s_1 , wobei die höherwertige Ziffer immer links steht ($s_1 s_0$).

Ein Word hat 4 hexadezimale Stellen s_0 bis s_3 , wobei es üblich ist, die Stellen **von rechts nach links** ($s_3 s_2 s_1 s_0$) zu schreiben.

2.2. FORMATE

Die Bytes werden im **Big-Endian** Format **von rechts nach links** und im **Little-Endian** Format **von links nach rechts** geschrieben.

Big-Endian	Little-Endian												
<table border="1"> <tr> <td>1b</td><td>0b</td></tr> <tr> <td>CA_h</td><td>FE_h</td></tr> <tr> <td>i</td><td>$i + 1$</td></tr> </table>	1b	0b	CA _h	FE _h	i	$i + 1$	<table border="1"> <tr> <td>0b</td><td>1b</td></tr> <tr> <td>FE_h</td><td>CA_h</td></tr> <tr> <td>i</td><td>$i + 1$</td></tr> </table>	0b	1b	FE _h	CA _h	i	$i + 1$
1b	0b												
CA _h	FE _h												
i	$i + 1$												
0b	1b												
FE _h	CA _h												
i	$i + 1$												
Zahlen beginnen mit dem «grossen Ende», dem MSByte (Most Significant)	Zahlen beginnen mit dem «kleinen Ende», dem LSByte (Least Significant)												

3b	2b	1b	0b
87 _h	65 _h	43 _h	21 _h
i	$i + 1$	$i + 2$	$i + 3$

0b	1b	2b	3b
21 _h	43 _h	65 _h	87 _h
i	$i + 1$	$i + 2$	$i + 3$

Vorteil von Little-Endian ist, dass man ein DWord mit Nullen in den höherwertigen Stellen als Word oder Byte mit derselben Adresse verwenden kann.

TODO:

diagram W2 S20

3. ASSEMBLER

Der Assembler ist ein Programm, das textuelle Befehle in Maschinencodes übersetzt. Konventionen hängen massgeblich vom Hersteller des Assemblers ab, selbst für denselben Prozessor. Wir verwenden in diesem Modul den Netwide Assembler [NASM](#).

3.1. SPEZIFIKATION VON DATEN

Quellcode	Resultat im Binär-Output
<code>db 48</code>	das Byte 48d
<code>db 0x35, 0h21, 049h</code>	die drei Byte 35h, 21h, 49h
<code>db 'a'</code>	das Byte mit dem ASCII-Code von a = 61h ≡ <code>db 0x61</code>
<code>db 'Hallo'</code>	die ASCII-Codes von H, a, l, l und o ≡ <code>db 0x48, 0x61, 0x6c, 0x6c, 0x6f</code>

Statt einzelnen Bytes können auch Words, DWords und QWords spezifiziert werden, Endianness muss dann zwingend beachtet werden:

`dw 0x2135`

`dd 0x2135`

`dq 0x2135`

TODO:

3.1.1. Spezifikation von Zahlen

\$ Präfix auch möglich.

Base	Suffix	Präfix
Hex	H / X	0h / 0x
Dec	D / T	0d / 0t
Oct	Q / O	0q / 0o
Bin	B / Y	0b / 0y

3.2. BITS

NASM übersetzt fast jede Anweisung direkt in Binärzahlen und schreibt diese als Sequenz von Bytes in die Zielfeile

```
nop          ; 90h
not rax      ; 48h F7h D0h
mov rax, rbx ; 48h 89h D8h
```

Traditionell arbeitet der NASM im historischen 16-Bit-Modus, Operationen haben hier andere Opcodes als im 64-Bit-Modus. Der 64-Bit-Modus wird mit der Direktive **BITS 64** aktiviert (möglichst als 1. Zeile)

3.3. OFFSET

Nummeriert man die Bytes in der Ausgabe-Sequenz fortlaufend, erhält jedes Byte einen **Offset** (Adresse, Index).

```
...
dd 0x12345678
...
```

78 _h	56 _h	34 _h	12 _h
i	$i + 1$	$i + 2$	$i + 3$

\$ bezeichnet den aktuellen Offset, bevor die von der aktuellen Zeile generierten Bytes in die Ausgabe-Sequenz geschrieben werden.

```
db 'BSys 1?!'
dq $
; erzeugt          42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00
```

3.4. LABELS

Am Anfang jeder Zeile in der Quelle kann ein Label stehen, das **nicht** in Maschinencode übersetzt wird:

```
my_text: db 'BSys 1?!'
current: dq current
; erzeugt auch          42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00
```

Intern assoziiert der Assembler das Label w mit dem Offset $A(w)$ des nachfolgenden Befehls in der Ausgabefeile und ersetzt jede Verwendung von w in der Quelle durch $A(w)$ in der Ausgabe-Sequenz.

42 _h	53 _h	79 _h	73 _h	20 _h	31 _h	3f _h	21 _h	08 _h	00 _h	00 _h	00 _h	00 _h	00 _h	00 _h	00 _h
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
^ my_text								^ current							

NASM kann während der Übersetzung einfache Arithmetik ausführen:

```
length: dq after_my_text - my_text
my_text: db 'BSys 1?!'
after_my_text:
; erzeugt          08 00 00 00 00 00 00 00 42 53 79 73 20 31 3f 21
```

3.5. FLAT-FORM BINARIES

Per default erzeugt der NASM sogenannte *Flat-Form Binaries* (plain/pure/raw): Die reine Bytesequenz analog zum Quell-Text ohne Zusatzinformationen.

3.6. OBJEKT-DATEIEN

Objekt-Dateien enthalten darüber hinaus noch weitere Informationen, die später benötigt werden, u.a. die Symboltabelle.

Exportierte Symbole: mit `global` deklarierte Label

Importierte Symbole: mit `extern` deklarierte Label, haben keinen definierten Wert innerhalb der Objekt-Datei

Auf 64-Bit Linux sind Objekt-Dateien im Format ELF64 mit Endung `*.o`

```
nasm -f elf64 prog.asm -o prog.o
```

ELF64 kann mit dem Tool `objdump` analysiert werden:

```
objdump -t -d -Mintel prog.o
```

3.6.1. Symboltabelle

Die Symboltabelle enthält einen Eintrag pro Symbol.

```
global x, y
extern z
dq 0, 0
w: dq 0x12345678
x: dq 0xCAFEFACE
y: dq z
```

Offset	Attribute (g)lobal, (l)ocal	Sektion	Zusatzinformationen	Name
000000000000000010	l	.text	0000000000000000	w
000000000000000000		*UND*	0000000000000000	z
000000000000000018	g	.text	0000000000000000	x
000000000000000020	g	.text	0000000000000000	y

Das als extern deklarierte Label `z` erscheint als `*UND*` (undefined)